CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

Aave v3 Pool (PoolInstance)

0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2

Ethereum

Compiler: v0.8.27+commit.40a35a09

Date: October 20, 2026

OVERALL RISK RATING

High

Manual review of verified source. A risk snapshot, not a guarantee.

Executive Summary

Independent review of the Aave v3 Pool implementation at 0x728a138a4823392c2efa55e028d434f526fe03cf, the logic contract behind the Pool proxy at 0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2. 42 source files, 9,139 lines, verified on Etherscan, compiled with solc 0.8.27. This is the current generation of the contract, carrying the position manager delegation and Umbrella deficit mechanics introduced after v3.3.

No exploitable flaw was found in the supply, borrow, repay, flash loan or liquidation arithmetic. The accounting is careful, the health factor is checked on every path that can worsen a position, and the library split keeps each operation legible. The code is good.

The risk is entirely in authority, and this review rates it High on that basis alone, in line with how a single owner key over a Chainlink aggregator proxy is rated. Two privileged functions reachable only by the PoolConfigurator can change the economics of every open position in the same block they are called, with no bound expressed anywhere in the Pool itself. Reserve configuration, which includes the liquidation threshold that decides who is liquidatable, is assigned as a raw packed word with no validation at this layer. The liquidation grace period accepts any future timestamp with no ceiling, and while it is set, liquidation is blocked for every position that touches the reserve on either the collateral or the debt side while interest keeps accruing.

Neither is a bug. Both are deliberate, both sit behind Aave governance, and Aave governance is among the more credible in the industry. The point of recording them is narrower: the Pool contract enforces none of it. Every bound lives in a separate upgradeable contract reached through the addresses provider, so what protects a depositor is the continued correctness of a contract this one does not verify. That is the whole of the exposure, and a depositor reading only the Pool cannot see it.

Findings (5)

AAVE-01 · No SWC mapping (centralization)

High

Reserve configuration is assigned as an unvalidated word and takes effect in the same block

DESCRIPTION

setConfiguration takes a packed ReserveConfigurationMap and writes it straight into storage. The only checks are that the asset is non-zero and listed. Nothing at this layer validates the contents of the word.

That word carries the loan to value ratio, the liquidation threshold, the liquidation bonus, the reserve factor, the decimals, the supply and borrow caps, the debt ceiling, and the active, frozen, paused and borrowable bits. A single call replaces all of it at once, and the new values apply to positions that already exist from the moment the transaction lands.

The liquidation threshold is the consequential one. It is the number that decides whether an open position is healthy or liquidatable. Lowering it does not warn anyone, does not phase in, and gives no exit window. Positions that were comfortably collateralised before the transaction are liquidatable in the block after it, and

liquidators compete for them in the ordinary way.

Validation of these values does exist, in PoolConfigurator. It does not exist here.
PoolConfigurator is a separate contract, separately upgradeable, resolved at call time through ADDRESSES_PROVIDER.getPoolConfigurator(). The Pool accepts whatever address currently answers to that name and whatever word it sends.

IMPACT

Whoever holds the PoolConfigurator role can make any set of existing positions liquidatable in one transaction, with no delay and no notice to the affected users. In the worst case, a compromised or replaced configurator can do this across every reserve. No user action can prevent it and no user is warned before it happens. Depositors reading the Pool source cannot determine what bounds apply, because the bounds are not in this contract.

RECOMMENDATION

Express the invariants that matter in the Pool rather than only in the configurator, so they survive a configurator change. At minimum, reject a configuration where loan to value exceeds the liquidation threshold, and reject a liquidation threshold decrease beyond a fixed maximum step per call. Better still, make a liquidation threshold decrease take effect only after a timelock measured in days, which gives borrowers the chance to add collateral or unwind. A decrease that must be announced before it binds costs the protocol nothing in a normal risk update and removes the entire scenario above.

```
function setConfiguration(
address asset,
DataTypes.ReserveConfigurationMap calldata configuration
) external virtual override onlyPoolConfigurator {
require(asset != address(0), Errors.ZeroAddressNotValid());
require(_reserves[asset].id != 0 || _reservesList[0] == asset,
Errors.AssetNotListed());
_reserves[asset].configuration = configuration;
}
```

AAVE-02 · No SWC mapping (missing parameter bound)

Medium

Liquidation grace period accepts any future timestamp, with no ceiling

DESCRIPTION

setLiquidationGracePeriod accepts a uint40 timestamp and PoolLogic assigns it without examining it. There is no maximum, no bound relative to block.timestamp, and no check that the value is reasonable. A uint40 reaches the year 36812.

ValidationLogic then blocks liquidation while that timestamp has not passed, and it checks both sides of the position:

require(
collateralReserve.liquidationGracePeriodUntil < uint40(block.timestamp) &&
debtReserve.liquidationGracePeriodUntil < uint40(block.timestamp),
Errors.LiquidationGraceSentinelCheckFailed()
);

The two-sided check is what widens this. Setting the grace period on one reserve suspends liquidation not only for positions collateralised in that asset but also for every position anywhere in the pool that borrowed it.

One call over a widely borrowed asset reaches a large share of open positions.

The mechanism itself is sensible. It exists so that a reserve can be re-enabled after a pause without liquidating users who had no opportunity to react, which is a real problem worth solving. The absence of a ceiling is what turns a safety feature into an open-ended one.

IMPACT

A configurator can suspend liquidations indefinitely across a large portion of the pool. Interest continues to accrue on positions that cannot be closed, so positions that are merely underwater become deeply underwater, and the resulting bad debt is absorbed by depositors in that reserve rather than by the borrowers. The loss is not immediate and not obvious, which is what makes it worth recording: by the time it is visible it has already been taken.

RECOMMENDATION

Bound the parameter where it is set. Require that until is no more than a fixed maximum beyond block.timestamp, chosen to match the operational need the feature exists for, which is hours rather than years. A constant such as MAX_GRACE_PERIOD of 72 hours, enforced with require(until <= block.timestamp + MAX_GRACE_PERIOD), preserves every legitimate use and removes the unbounded case entirely. It is a one line change.

AAVE-03 · No SWC mapping (centralization)

Low

rescueTokens transfers any token to any address with no exclusion for listed assets

DESCRIPTION

rescueTokens is gated by onlyPoolAdmin and does nothing else. PoolLogic.executeRescueTokens is a bare transfer:

```
function executeRescueTokens(address token, address to, uint256 amount) external {
IERC20(token).safeTransfer(to, amount);
}
```

There is no check that the token is not the underlying of a listed reserve, no restriction on the destination, and no event beyond the ERC-20 Transfer.

The severity is Low rather than High because of a fact that has to be verified rather than assumed: the Pool does not custody user funds. Supplied underlying is transferred directly to the aToken, never to the Pool, which is visible at SupplyLogic.sol:62:

```
IERC20(params.asset).safeTransferFrom(params.user, reserveCache.aTokenAddress, params.amount);
```

So the balance this function can reach is only what has arrived at the Pool by accident. That is not nothing. Users send tokens to the wrong address in every protocol, and the Pool address is the one most users associate with Aave, so it accumulates stray balances precisely because it is the published address.

IMPACT

Tokens mistakenly sent to the Pool are recoverable only by the pool admin and can be sent anywhere, including to an address that is not the original sender. The function is a recovery tool with no obligation to return anything to whoever lost it. No user funds held by the protocol in normal operation are at risk.

RECOMMENDATION

Keep the function, it serves a real purpose, but narrow it. Emit a dedicated RescueTokens event carrying token, destination and amount, so recoveries are observable on chain rather than indistinguishable from any other transfer. Consider rejecting tokens that are the underlying or aToken of a listed reserve, which removes the only variant of this function that could ever touch protocol accounting.

AAVE-04 · Not applicable

Informational

Position manager delegation is correctly bounded, recorded as a reference implementation

DESCRIPTION

This review looks for authority that outlives a user's consent, so it is worth recording where that is done properly. The position manager mechanism added after v3.3 lets a third party act on a user's position, which is the shape of permission that usually goes wrong. Here it does not.

Four properties hold together. The grant is per user and per manager, stored as _positionManager[user][manager], so approving one address grants nothing to any other. The user can revoke at any time through approvePositionManager with approve false. The manager can also walk away unilaterally through renouncePositionManagerRole, which matters more than it appears: a delegate that cannot resign is a liability to both parties. And the surface is two functions, setUserUseReserveAsCollateralOnBehalfOf and setUserEModeOnBehalfOf, neither of which moves funds.

Most importantly, neither can be used to harm the user it acts for. Both paths end in a health factor check, validateHFAndLtvzero when collateral is disabled and validateHealthFactor after an eMode change. A position manager attempting to push a user into liquidation, in order to liquidate them, finds the transaction reverts.

IMPACT

None. This is noted as a positive finding. A delegation that is scoped, revocable from both sides, and validated against the interest of the account it acts for is the pattern most protocols get wrong, and it is the answer to question four of the Contract Layer Trust Checklist.

RECOMMENDATION

No change. Teams building delegation or session key mechanics should read this implementation before writing their own, in particular the decision to validate the health factor after the delegated action rather than trusting the delegate.

AAVE-05 · No SWC mapping (missing parameter bound)

Informational

Flash loan premium is set with no upper bound

DESCRIPTION

updateFlashloanPremium assigns _flashLoanPremium directly with no range check. The storage slot is a uint128 while the value is used as a basis point premium, so the accepted range is many orders of magnitude wider than any meaningful premium.

The practical consequence is bounded, which is why this is informational rather than a finding with teeth. An absurd premium makes flash loans revert or become uneconomic, which is a denial of a feature rather than a loss of funds, and flash loan borrowers see the premium before they commit. It is recorded because it is

the same shape as AAVE-02, a parameter with no ceiling, and because the two together describe the house style of this contract: validation lives in the configurator, not here.

IMPACT

A configurator can disable flash loans in practice by setting an uneconomic premium. No user funds are at risk and no existing position is affected.

RECOMMENDATION

Add require(flashLoanPremium <= MAX_FLASHLOAN_PREMIUM) with a constant in the low hundreds of basis points. The check costs one comparison and makes the contract self describing about what values are intended.

Appendix · Metadata

Contract	Aave v3 Pool (PoolInstance)
Address	0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2
Chain	Ethereum
Compiler	v0.8.27+commit.40a35a09
Audit ID	aave-v3-pool-2026-10-20
Date	10/20/2026

Files analyzed

```
[dependency] openzeppelin-contracts/contracts/token/ERC20/IERC20.sol
[dependency] openzeppelin-contracts/contracts/utils/Address.sol
[dependency] openzeppelin-contracts/contracts/utils/Context.sol
[dependency] openzeppelin-contracts/contracts/utils/Errors.sol
[dependency] openzeppelin-contracts/contracts/utils/Multicall.sol
[dependency] openzeppelin-contracts/contracts/utils/math/SafeCast.sol
[dependency] src/contracts/dependencies/gnosis/contracts/GPv2SafeERC20.sol
[user] src/contracts/dependencies/openzeppelin/contracts/Address.sol
[user] src/contracts/instances/PoolInstance.sol
[user] src/contracts/interfaces/IACLManager.sol
[user] src/contracts/interfaces/IAToken.sol
[user] src/contracts/interfaces/IERC20WithPermit.sol
[user] src/contracts/interfaces/IInitializableAToken.sol
[user] src/contracts/interfaces/IInitializableDebtToken.sol
[user] src/contracts/interfaces/IPool.sol
[user] src/contracts/interfaces/IPoolAddressesProvider.sol
[user] src/contracts/interfaces/IPriceOracleGetter.sol
[user] src/contracts/interfaces/IReserveInterestRateStrategy.sol
[user] src/contracts/interfaces/IScaledBalanceToken.sol
[user] src/contracts/interfaces/IVariableDebtToken.sol
[user] src/contracts/misc/aave-upgradeability/VersionedInitializable.sol
[user] src/contracts/misc/flashloan/interfaces/IFlashLoanReceiver.sol
[user] src/contracts/misc/flashloan/interfaces/IFlashLoanSimpleReceiver.sol
[user] src/contracts/protocol/libraries/configuration/EModeConfiguration.sol
[user] src/contracts/protocol/libraries/configuration/ReserveConfiguration.sol
[user] src/contracts/protocol/libraries/configuration/UserConfiguration.sol
[user] src/contracts/protocol/libraries/helpers/Errors.sol
[user] src/contracts/protocol/libraries/helpers/TokenMath.sol
[user] src/contracts/protocol/libraries/logic/BorrowLogic.sol
[user] src/contracts/protocol/libraries/logic/FlashLoanLogic.sol
[user] src/contracts/protocol/libraries/logic/GenericLogic.sol
[user] src/contracts/protocol/libraries/logic/LiquidationLogic.sol
[user] src/contracts/protocol/libraries/logic/PoolLogic.sol
[user] src/contracts/protocol/libraries/logic/ReserveLogic.sol
[user] src/contracts/protocol/libraries/logic/SupplyLogic.sol
[user] src/contracts/protocol/libraries/logic/ValidationLogic.sol
[user] src/contracts/protocol/libraries/math/MathUtils.sol
```

```
[user] src/contracts/protocol/libraries/math/PercentageMath.sol
[user] src/contracts/protocol/libraries/math/WadRayMath.sol
[user] src/contracts/protocol/libraries/types/DataTypes.sol
[user] src/contracts/protocol/pool/Pool.sol
[user] src/contracts/protocol/pool/PoolStorage.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

Request Manual Audit ’

`https://cryptohawking.com/audit/manual?ref=aave-v3-pool-2026-10-20`

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

`https://cryptohawking.com/services`

`WhatsApp (preferred): +92 322 4274236, https://wa.me/923224274236`